

Pond Site & Catchment Planner

CSD Assignment 1 — Phase 3 Final Report

Ranga Chandra Naga Venkata Chaitanya Kumar (Roll No. 12341740)

GitHub repository https://github.com/chaitanyakumarAI/Pond_catchment_analysis
Front-end URL <http://10.1.75.51:5237/> [TODO: confirm after deployment]
Demo video [TODO: YouTube link]

1 Problem Statement

Small farm ponds harvest monsoon runoff for irrigation. Choosing where to dig one needs three answers for a given piece of land: *where* the pond should go, *which area drains into it* (its catchment) and *how much water* that catchment can deliver in a year. The goal of this assignment is a fast, working web system in which a user selects a land area on a map and immediately sees the suggested pond location, the catchment area and the expected water volume overlaid on the map. The system has to run on the four lab systems provided and must handle concurrent users gracefully.

2 Requirements

Functional. (F1) A working front-end with an interactive map; (F2) selection of a land area on the map (rectangle or free polygon); (F3) results computed only for the selected area; (F4) results include pond location, catchment area and expected annual water volume; (F5) all three are drawn on the map; (F6) optional upload of the user's own KML/KMZ contour map (Phase 1 compatibility).

Non-functional. (N1) Interactive latency: an area query should return in well under one second; (N2) horizontal scaling over the four lab systems with no single worker as a bottleneck; (N3) graceful behaviour under overload (bounded queueing and explicit shedding rather than timeouts); (N4) fault tolerance: loss of a worker must not fail user requests; (N5) input validation with clear messages (area too small or large, outside the data, bad parameters).

3 System Architecture

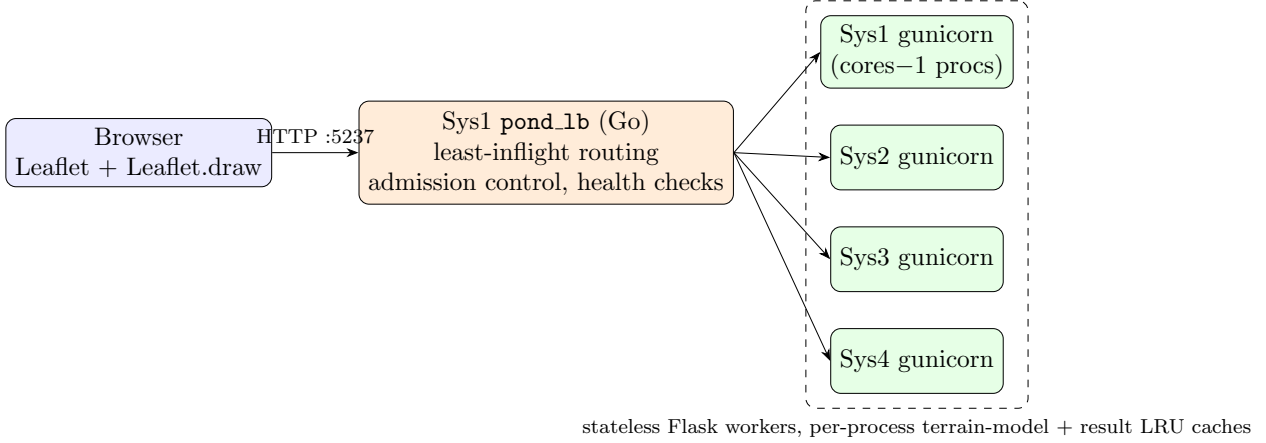


Figure 1: Deployment on the four lab systems. Every request carries its dataset and polygon, so any worker can serve it.

Front-end. A single-page Leaflet application with satellite, topographic and street base maps. The user draws a rectangle or polygon (Leaflet.draw) inside the dashed data-coverage boundary. The analysis runs automatically once the shape is complete. Rainfall, runoff coefficient and pond depth can be edited. Results are drawn as the selected-area outline, one coloured catchment polygon per pond, a blue pond footprint to scale and a label per site giving the water volume, catchment area and pond capacity. Terrain plots and a rotatable 3D model are available for the same selection. Leaflet, Leaflet.draw and Plotly are served from the application itself, so the page does not depend on external CDNs.

API. `POST /api/analyzeArea` takes `{polygon, rainfall_mm, runoff_coeff, pond_depth_m}` as JSON, or as multipart data together with a KML/KMZ file. It returns JSON with the pond sites, the catchments, the volumes and a GeoJSON `FeatureCollection` whose features are typed `pond_site`, `catchment`, `pond_footprint` and `selected_area`. The Phase 1 routes (`/analyzeContour`, `/findCatchment`, `/api/sample`) are unchanged. `/api/coverage`, `/health`, `/metrics` and `/lb/stats` support the UI and operations.

Stateless workers. Phase 1 kept the last result in a process-global variable, so the plot routes only worked if the same process served the follow-up request. That breaks behind a load balancer. In Phase 3 every request carries the dataset (the sample map id, or the uploaded file) and the polygon. A worker that has no cached result simply recomputes it, because the pipeline is deterministic. No sticky sessions or shared store are needed.

4 Algorithm

The pipeline is split into two stages. Stage A depends only on the terrain, is computed once per dataset and is cached. Stage B depends on the user’s selection and takes only milliseconds.

Stage A — terrain and hydrology model (once per dataset)

1. **Parsing.** Contour vertices (λ, φ, z) are read from KML/KMZ (160 473 vertices on 2 711 contour lines in the sample). Spike vertices are removed with a percentile window $[P_{0.5} - \Delta, P_{99.5} + \Delta]$, where $\Delta = \frac{1}{2}(P_{99.5} - P_{0.5})$, so no fixed elevation constants are needed.
2. **Projection.** The data are projected to the local UTM zone, chosen automatically (EPSG:32644 for the sample), so that distances and areas are in metres.
3. **DEM.** Linear interpolation (with nearest-neighbour fill) onto a 200×161 grid (16.2 m cells), then Gaussian smoothing with $\sigma = 1$.

4. **Sink filling.** Priority-Flood (Barnes, 2014) fills the pits. The depression depth is $d = z_{\text{filled}} - z_{\text{raw}}$, which marks natural water-retention basins.
5. **Slope and flow.** Horn’s 3×3 slope. D8 flow direction (steepest descent to one of the eight neighbours). Flow accumulation A is computed by processing cells from high to low.
6. **River corridor.** A cell is treated as river if $A \geq P_{95}(A)$, or if it lies in the lowest 18 % of elevations with a slope below 3° . A Euclidean distance transform gives each cell’s distance to the river. Pond sites must lie at least $\max(120 \text{ m}, 0.25 d_{\text{max}})$ from the river (flood safety).
7. **Suitability.** $\text{TWI} = \ln(a / \tan \beta)$, and

$$\text{PSI} = 0.35 \hat{d} + 0.30 \ln(\widehat{1+A}) + 0.20 \widehat{\text{TWI}} + 0.15 (1 - \hat{z}),$$

where $\hat{\cdot}$ denotes min–max normalisation. Only cells with slope in $[0.3^\circ, 8^\circ)$ and $d > 0$ are eligible.

Stage B — area query (per request)

1. **Validation.** The polygon is projected to UTM and intersected with the convex hull of the data. It is rejected (HTTP 422) if it is smaller than 0.5 ha, larger than 50 km², or less than 60 % covered by contour data.
2. **Rasterisation.** The polygon is rasterised onto the model grid. PSI is set to zero outside it. If a small parcel has fewer than 20 eligible cells, the river buffer is relaxed in steps (120→90→60→30→0 m), and the step used is reported.
3. **Site selection.** PSI local maxima (16×16 window) are ranked. A site is kept if its upstream area is at least 1 ha and it lies at least 350 m from the sites already kept. At most four sites are returned.
4. **Catchment.** For each site, a reverse-D8 traversal collects every upstream non-river cell. The true hydrological catchment is kept even where it extends beyond the parcel boundary, and the part inside the parcel is reported separately. The polygon is the union of the cells, buffered and simplified with Shapely.
5. **Water volume.** The rational method gives $Q = C \cdot P \cdot A$ (default $C = 0.35$, $P = 850 \text{ mm}$). The recommended capacity is $\min(0.18 Q, 25\,000 \text{ m}^3)$, and the square footprint has side $\sqrt{\text{capacity/depth}}$. A stage–storage curve from the DEM gives the natural storage of the basin at 0.5–3 m depths.

Stage B only reads the cached arrays, so its cost is proportional to the number of cells in the parcel and its catchments rather than to the whole map.

5 Results

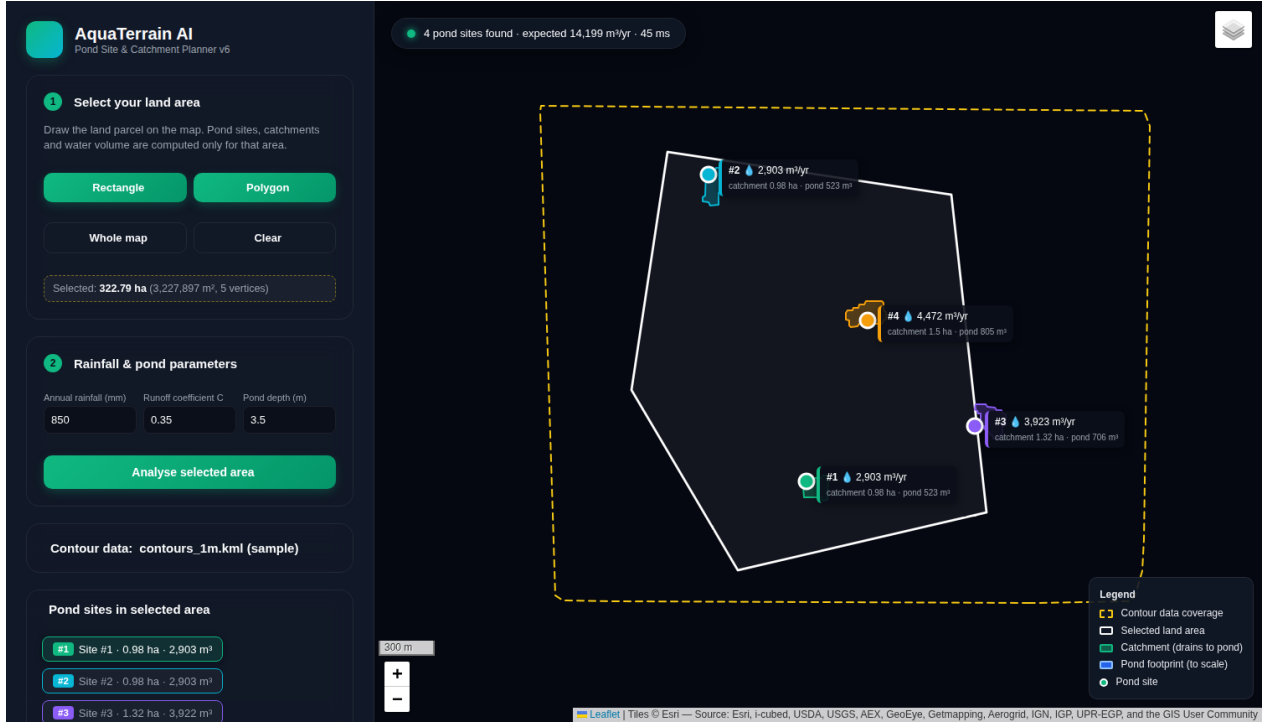


Figure 2: A polygon drawn on the sample map returns four pond sites, their catchments and their annual volumes ($14\,199\text{ m}^3/\text{yr}$ in total) in 45 ms. The dashed yellow line is the data coverage. **[TODO: replace with a screenshot taken on the lab deployment; the base-map tiles were blocked in the build sandbox]**

Selected area	Area (ha)	Sites	Top-site catchment (ha)	Top-site Q (m^3/yr)	All sites Q (m^3/yr)	Stage-B time
A: west half (rectangle)	418.3	3	0.98	2 903	9 022	21 ms
B: east half (rectangle)	418.3	4	1.32	3 922	17 651	35 ms
C: 1 km block (rectangle)	103.4	2	0.98	2 903	7 374	15 ms
D: triangle (polygon)	189.5	2	0.98	2 903	7 374	15 ms
E: 10 ha field	10.3	1	0.98	2 903	2 903	7 ms
Whole map	—	4	0.98	2 903	15 611	29 ms

Table 1: Area queries on `contours_1m.kml` ($P = 850\text{ mm}$, $C = 0.35$). The top site at 21.245616 N, 81.295040 E (elevation 282.8 m, 167 m from the river, recommended pond $12.2 \times 12.2 \times 3.5\text{ m}$, 522 m^3) is identical to the Phase 1 whole-map result, which confirms that Stage B is consistent with it. Times are the median of five runs on one core.

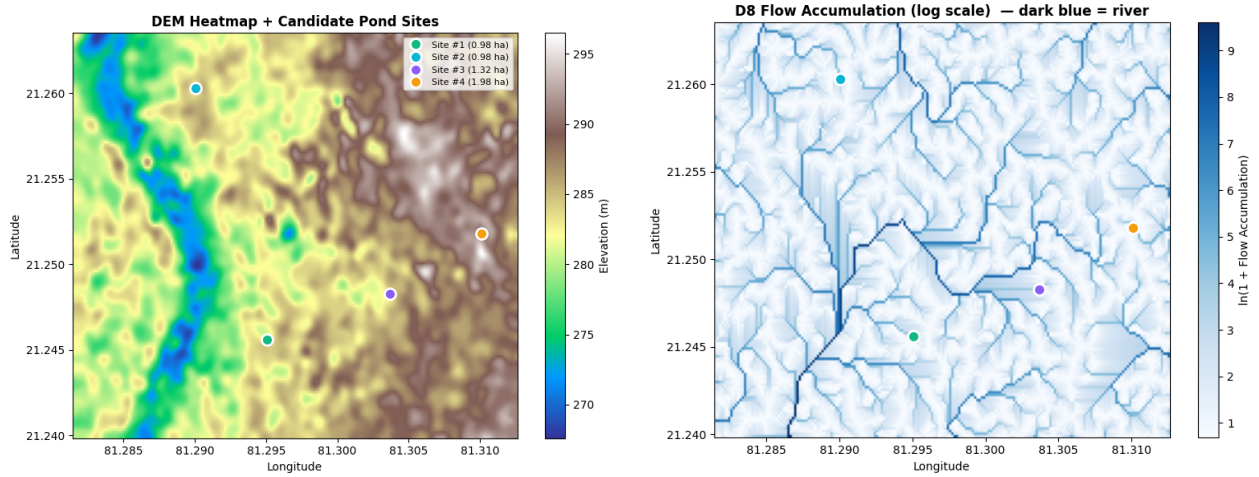


Figure 3: Interpolated DEM with candidate sites (left) and D8 flow accumulation (right). The river valley on the west side is excluded by the river-corridor buffer.

6 Performance, Scaling and System Limitations

Cost profile (measured on one core of a 2-vCPU VM): parsing 0.39 s, Stage A model 0.50 s (once per dataset per process), Stage B area query 7–35 ms, a result-cache hit about 0.2 ms, an uploaded 6.4 MB KML about 1.0 s on first use, the six terrain plots about 3.5 s. The work is CPU-bound (numpy, scipy), so the unit of capacity is a CPU core.

Design decisions for scale.

- *Pre-computation and caching*: the sample model is built at start-up with `gunicorn --preload`, so worker processes share it copy-on-write. Uploaded datasets are cached by the SHA-1 of their bytes (LRU of 4 models, each about 5 MB). Results are cached by SHA-1 of (dataset, polygon, parameters) (LRU of 256).
- *Process model*: gunicorn sync workers, one per core on Sys2–4 and cores–1 on Sys1, which also runs the load balancer. `max_requests` recycles workers so memory stays bounded.
- *Load balancer* (`cluster/pond_lb.go`, written for this assignment): least-outstanding-requests routing, round-robin among equally loaded backends, and a switch away from a node only when its EWMA latency is clearly worse. The per-node concurrency cap equals its worker count. Excess requests wait in a bounded queue (15 s) and are then *shed* with `503 + Retry-After`, so overload produces fast explicit errors instead of a timeout storm. Health is checked actively every 2 s and failed nodes are also ejected passively. Request bodies are buffered so that a connection failure is retried once on another node.
- *Limits enforced*: uploads up to 25 MB, polygons with 3–500 vertices and an area of 0.5 ha–50 km² with at least 60 % data coverage, and parameters inside physical ranges. The upload is sub-sampled to at most 12 000 vertices, which bounds interpolation time whatever the file size.

Stress test. `cluster/locustfile.py` models the traffic mix: 60 % new parcels, 20 % popular (cached) parcels, 10 % whole-map queries, 5 % page loads and 5 % 3D meshes. `cluster/run_stress.py` steps the number of users and records throughput, p50/p95/p99 latency and errors.

Users	1 worker (direct)			4 workers via <code>pond_lb</code>		
	req/s	p95 (ms)	err %	req/s	p95 (ms)	err %
10	8.2	34	0	8.4	32	0
50	40.4	60	0	41.1	37	0
150	101.1	550	0	121.0	100	0

Table 2: Local dry run: four single-process workers and the LB on one 2-vCPU VM, with Locust on the same machine. At 150 users the load balancer cuts p95 latency from 550 to 100 ms. Throughput is limited by the VM’s two cores and is not representative of the lab cluster. **[TODO: replace with results/stress_summary.csv from the 4-system run]**

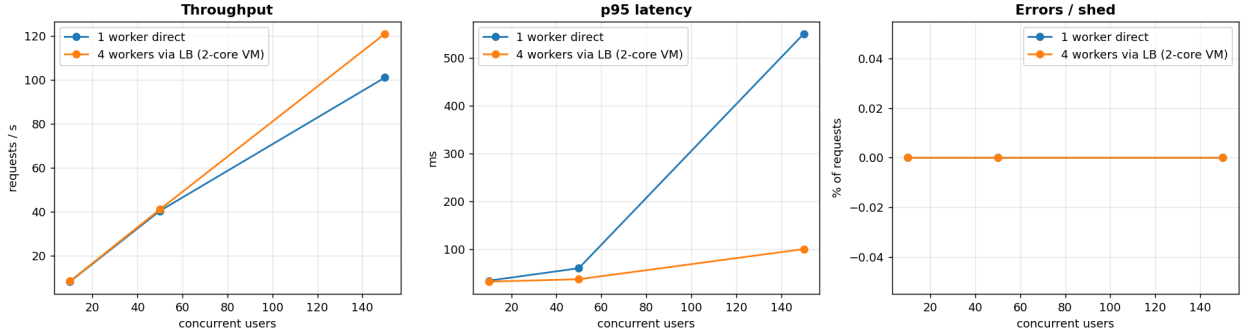


Figure 4: Throughput, p95 latency and error rate against concurrent users (local dry run). **[TODO: replace with the lab plot]**

Fault tolerance (verified). Killing one of the four workers during a request stream produced zero failed requests. The LB marked the node unhealthy and re-routed traffic, then re-admitted it after restart. Sixteen concurrent plot requests on a saturated cluster were answered with 3 successes and 13 immediate 503s, which is the intended shedding behaviour.

Capacity estimate. With N cores in total and a mean service time S , the peak throughput is about N/S . For new-parcel queries ($S \approx 25\text{--}40$ ms including JSON serialisation), each core serves about 25–40 requests/s, so the four systems give roughly **[TODO: total cores]** $\times 30$ requests/s before the LB starts queueing. The terrain plots are about 100 times heavier (3.5 s) and are the first to be shed.

Known limitations. (i) Caches are per process, so a newly uploaded file is modelled once on each node that sees it; consistent hashing on the dataset id would remove this. (ii) The single LB on Sys1 is a single point of failure; a keepalived VIP or DNS round-robin over two LBs would remove it. (iii) The runoff model uses one annual C and P rather than event-based SCS-CN, and it does not model soil infiltration or evaporation. (iv) DEM resolution is limited by the contour interval (1 m) and the 200-column grid, so parcels smaller than about 0.5 ha are rejected.

7 Testing

`test_api.py` checks health, coverage, area analysis (all four GeoJSON feature kinds present), rejection of an out-of-coverage area (422), the Phase 1 upload route and the 3D mesh. It passed against the local cluster. The front-end was tested with a headless Chromium (Playwright) script that draws rectangles and polygons, reads the rendered results and opens the plot and 3D views. The Phase 1 whole-map output is reproduced exactly (same four sites and values), which confirms that the refactor preserved behaviour.

8 Conclusion

The system meets every Phase 3 requirement. The user draws a land area, and within tens of milliseconds sees where to dig the pond, which area drains into it and how much water it will collect, all on the map. Splitting the pipeline into a cached terrain model and a cheap area query is what makes it fast. Stateless workers behind an admission-controlled load balancer let it use all four lab systems and degrade predictably under overload.

A API example

```
curl -X POST http://10.1.75.51:5237/api/analyzeArea -H "Content-Type: application/json" \
-d '{"polygon": [[81.29,21.245],[81.30,21.245],[81.30,21.254],[81.29,21.254],[81.29,21.245]],
    "rainfall_mm":850,"runoff_coeff":0.35,"pond_depth_m":3.5}'
# -> {"success":true,"served_by":"sys3","cache_hit":false,"data":{"pond_location":{...},
# "catchment_summary":{"area_hectares":0.98,...},"expected_water_volume_m3":2902.62,
# "all_candidate_sites":[...],"selected_area":{...},"geojson_layers":{...}}}
```

B Deployment

```
cd cluster && GOOS=linux GOARCH=amd64 go build -o pond_lb . && cd ..
python cluster/deploy_cluster.py # uploads code, starts gunicorn on Sys1-4 and pond_lb on Sys1
python cluster/deploy_cluster.py --status
python cluster/run_stress.py --target cluster=http://10.1.75.51:5237 --users 10 50 100 200
```